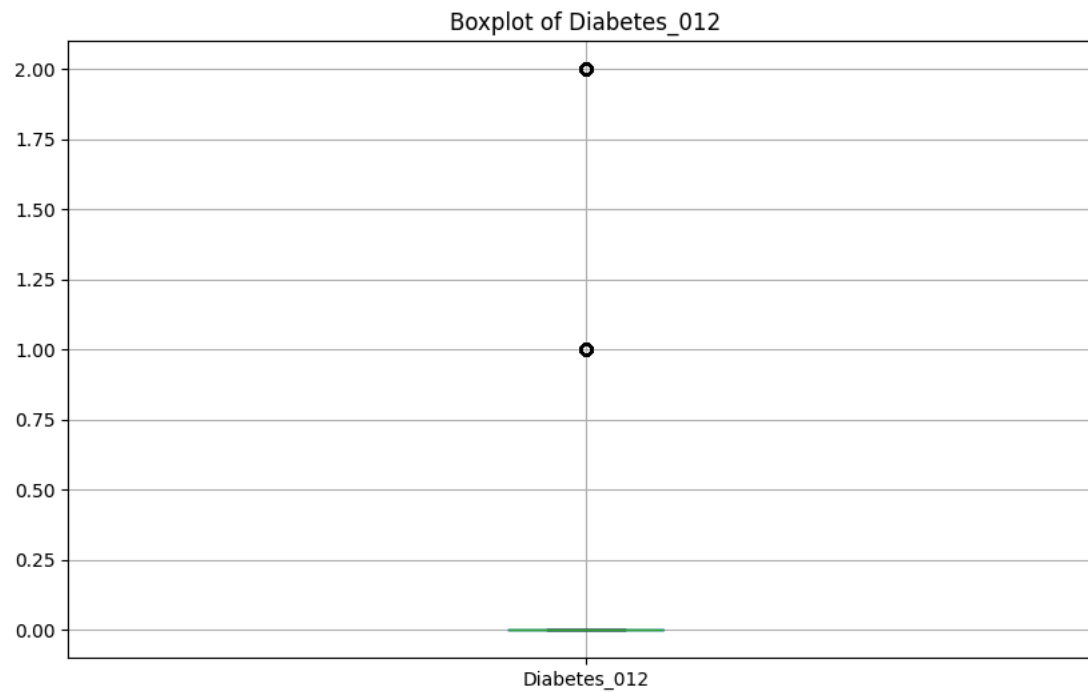


Important: Text generated by AI and may contain incorrect information, always check the content.
To identify outliers, we will examine the distribution of the numerical columns in the DataFrame.

```
```python
import pandas as pd
import matplotlib.pyplot as plt

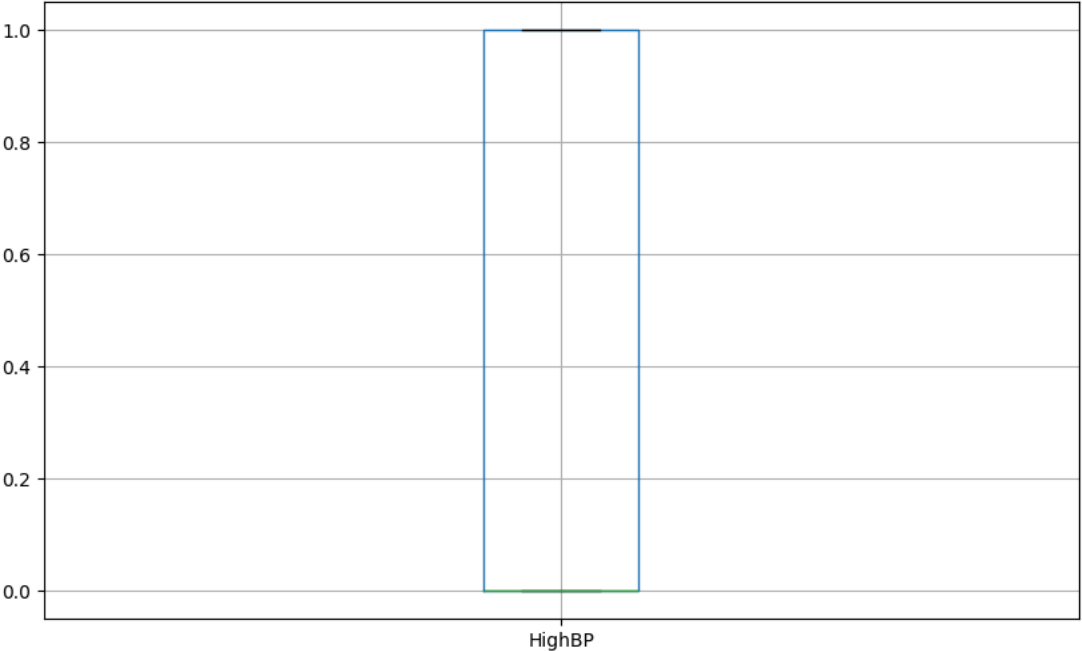
Boxplot for numerical columns to identify outliers
numerical_columns = df.select_dtypes(include=['float64']).columns

for column in numerical_columns:
 plt.figure(figsize=(10,6))
 df.boxplot(column=column)
 plt.title(f'Boxplot of {column}')
 plt.show()
```
```

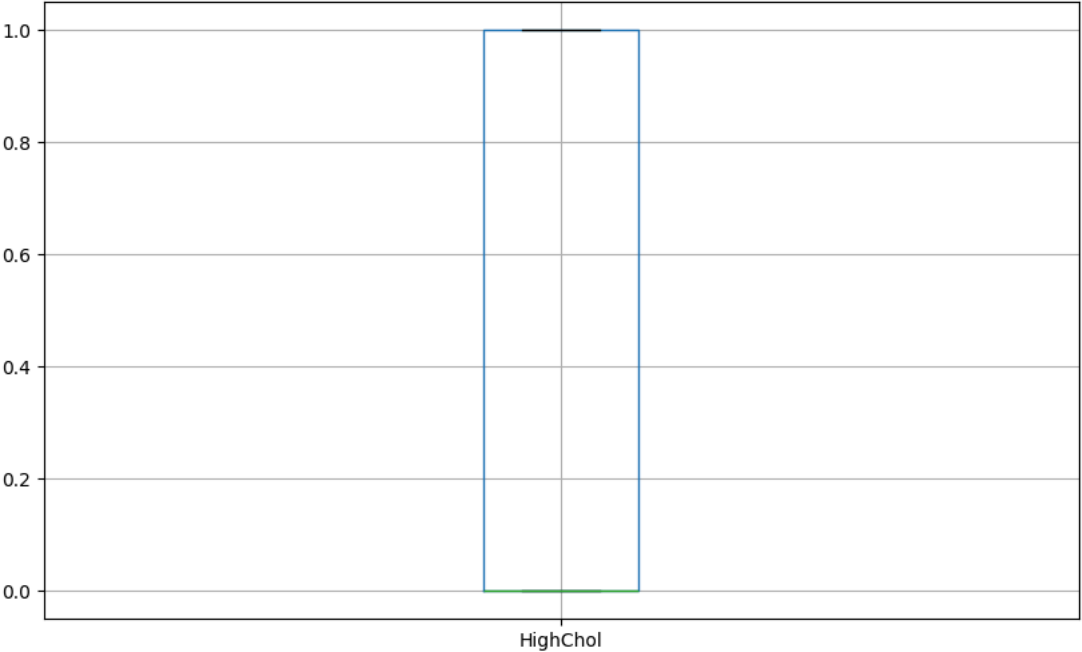




Boxplot of HighBP



Boxplot of HighChol

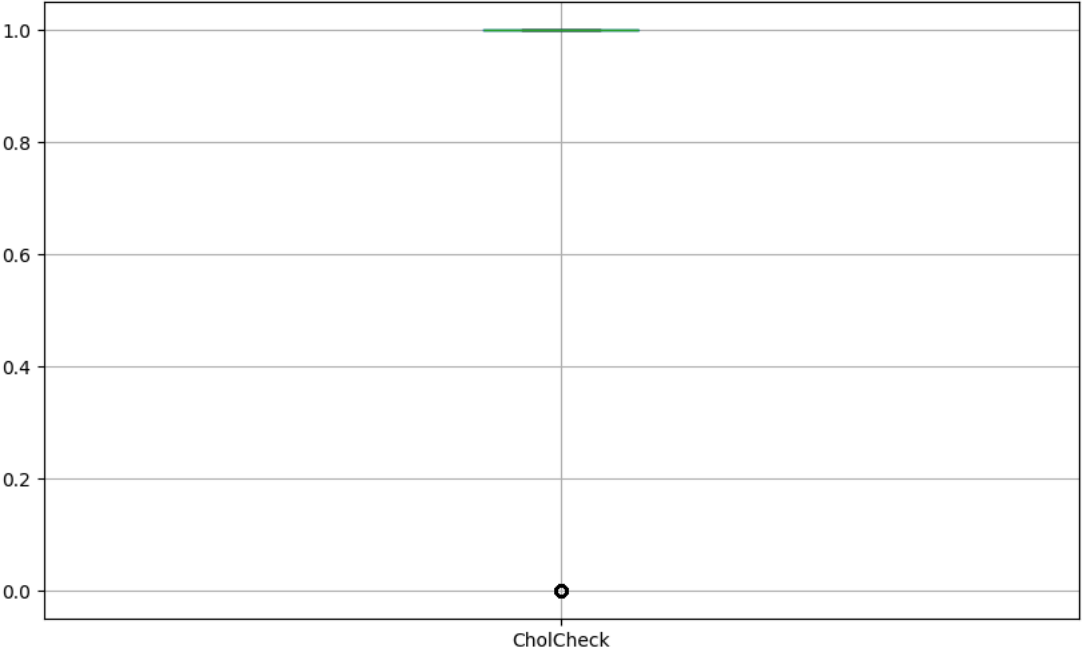


HighChol

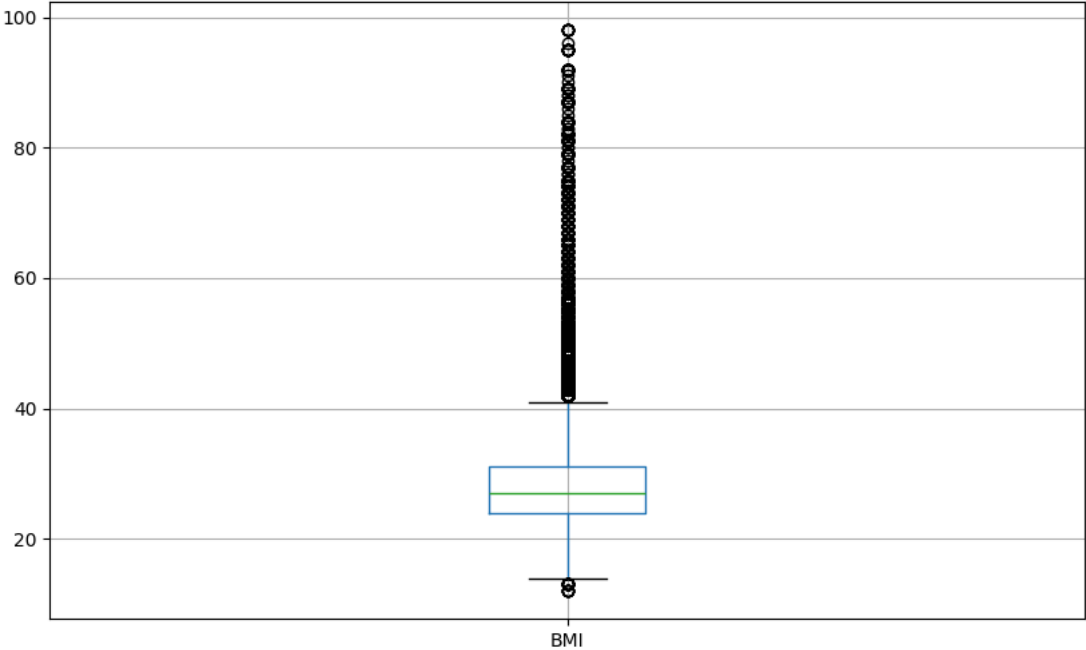




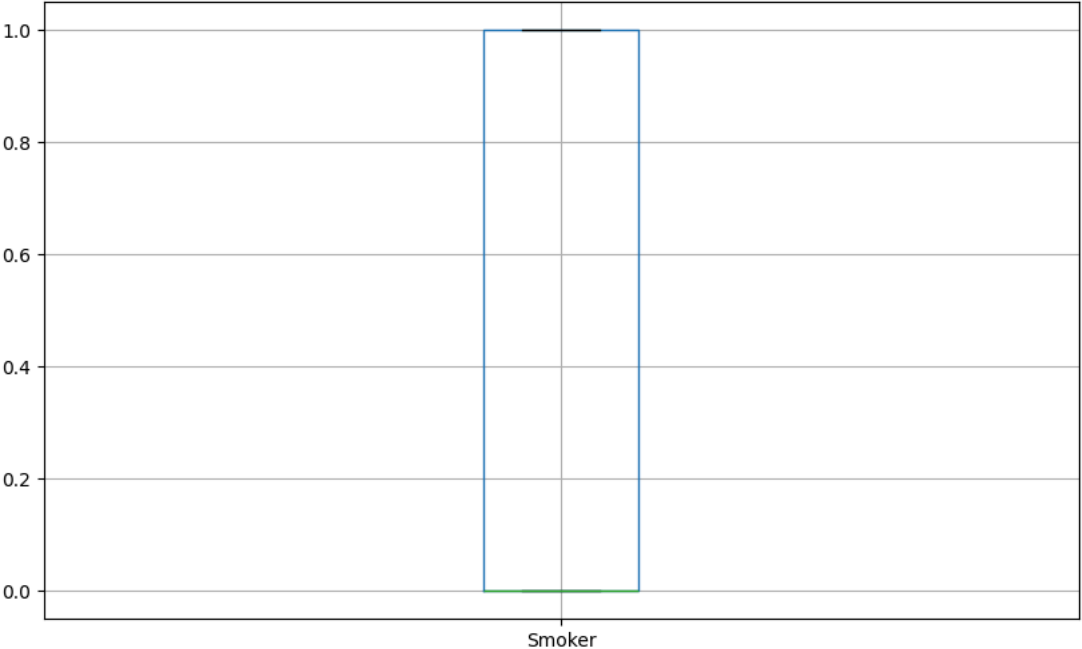
Boxplot of CholCheck



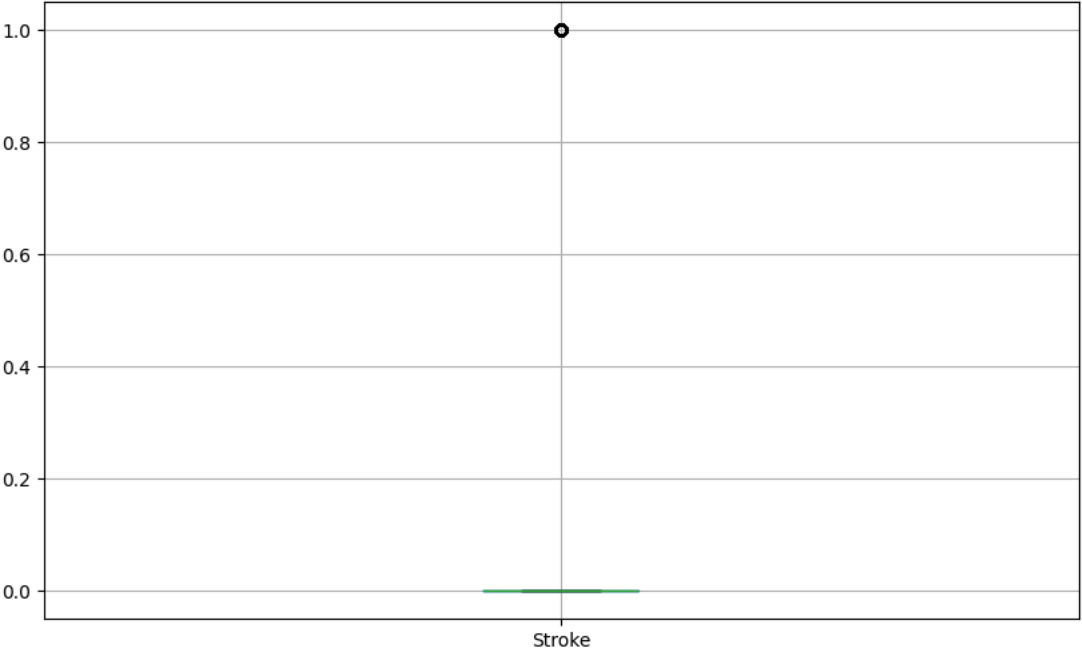
Boxplot of BMI



Boxplot of Smoker

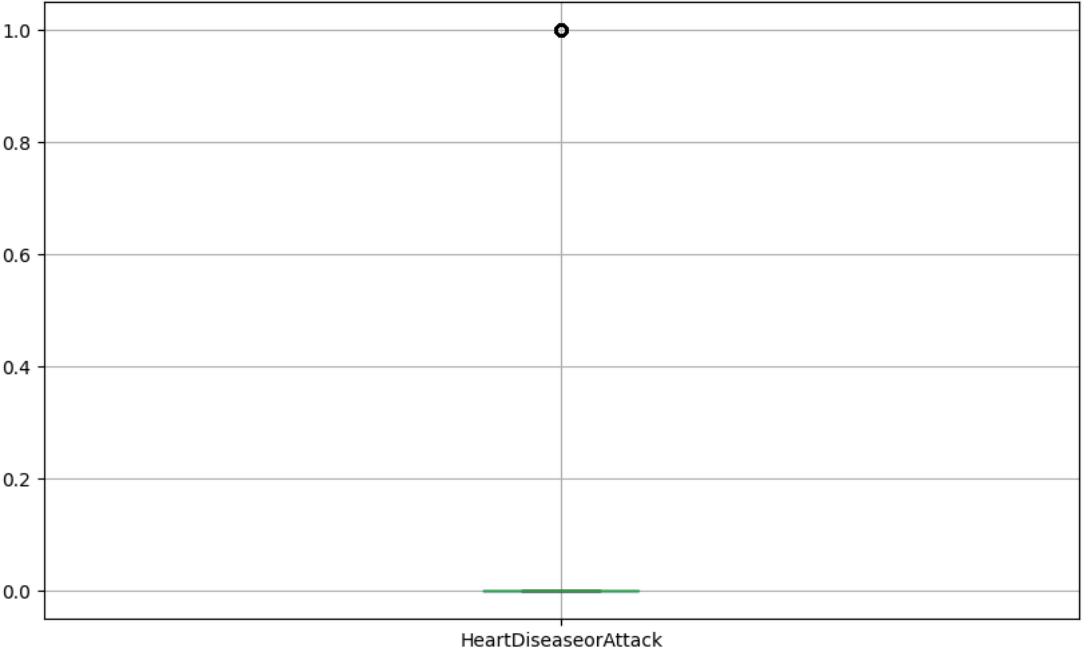


Boxplot of Stroke

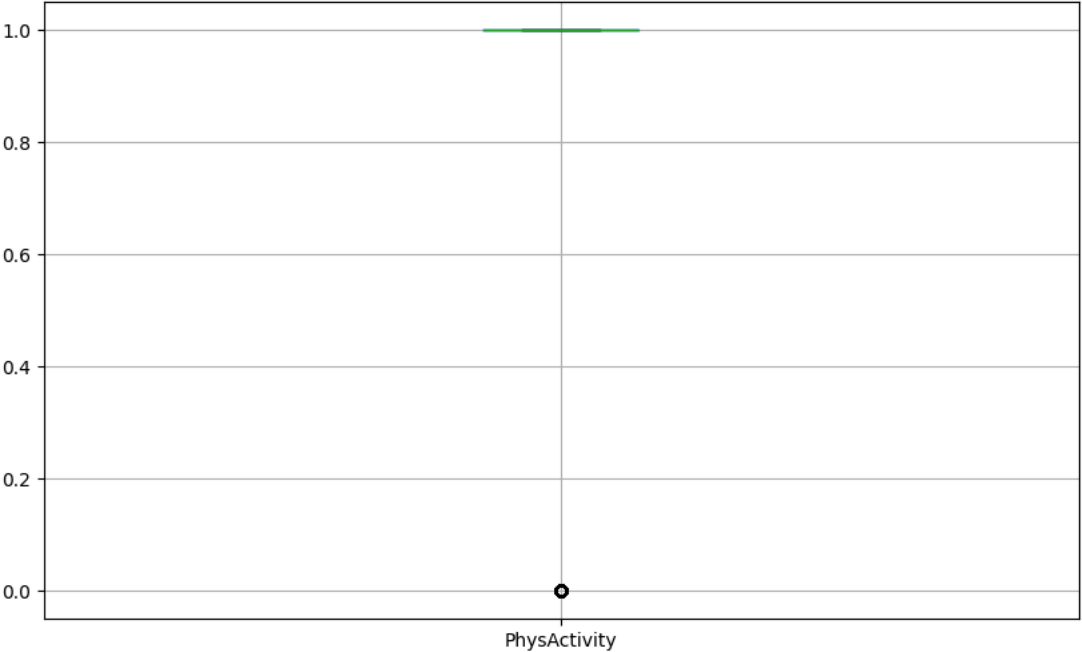




Boxplot of HeartDiseaseorAttack

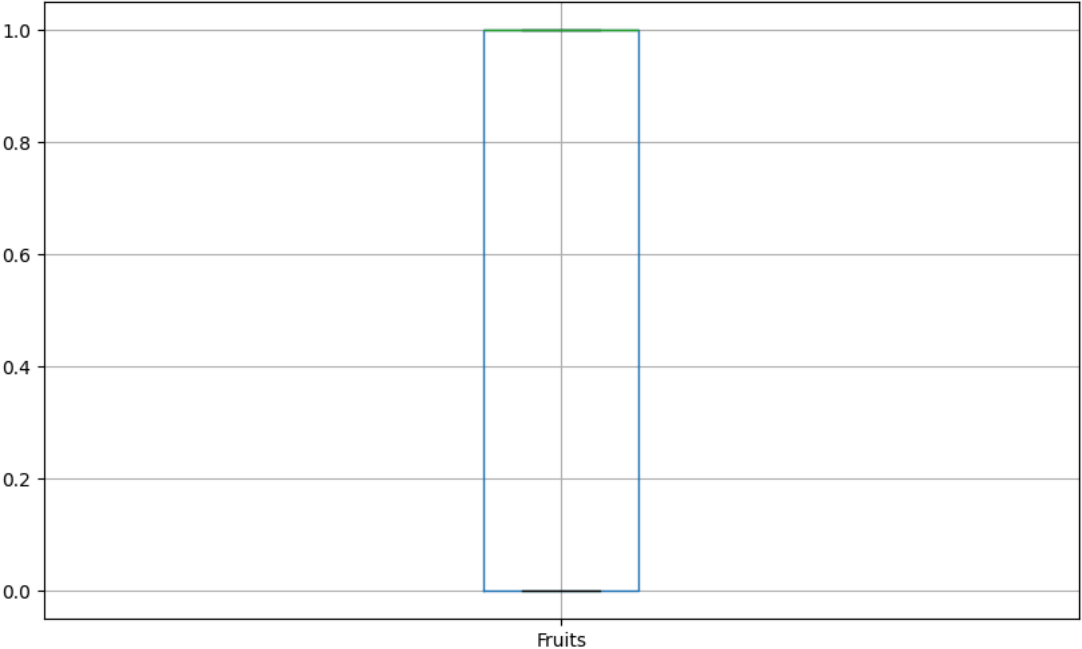


Boxplot of PhysActivity

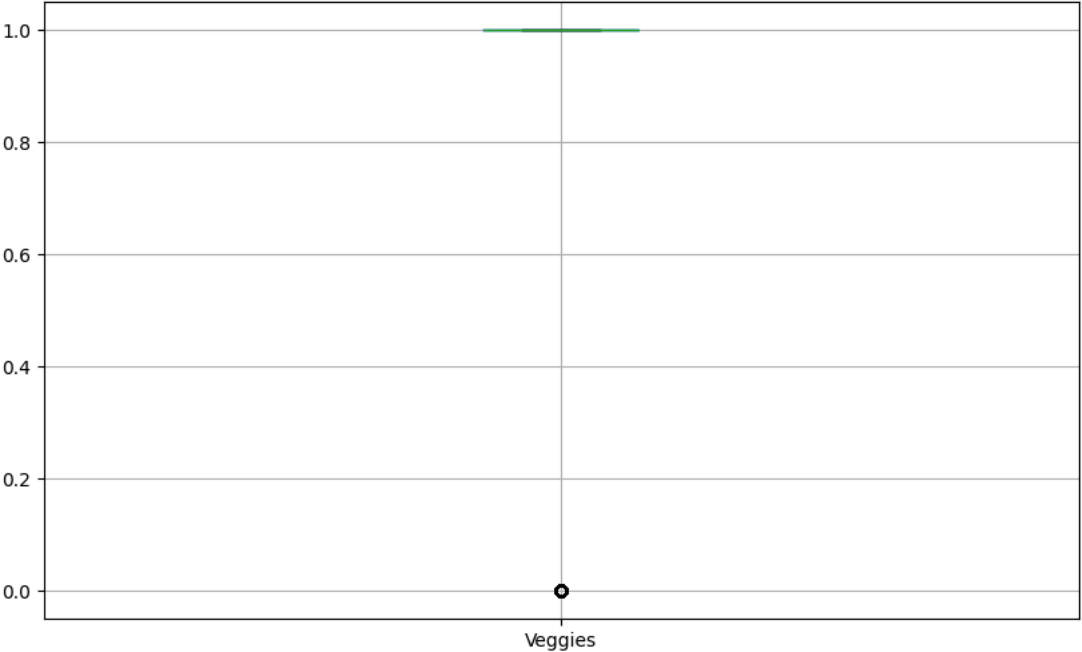




Boxplot of Fruits

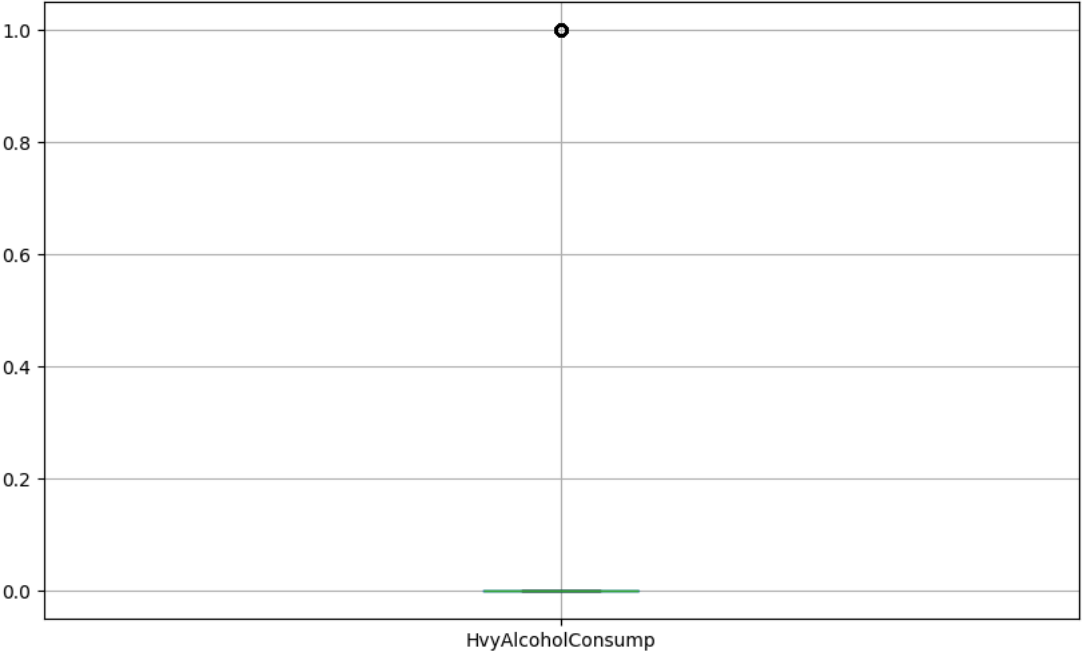


Boxplot of Veggies

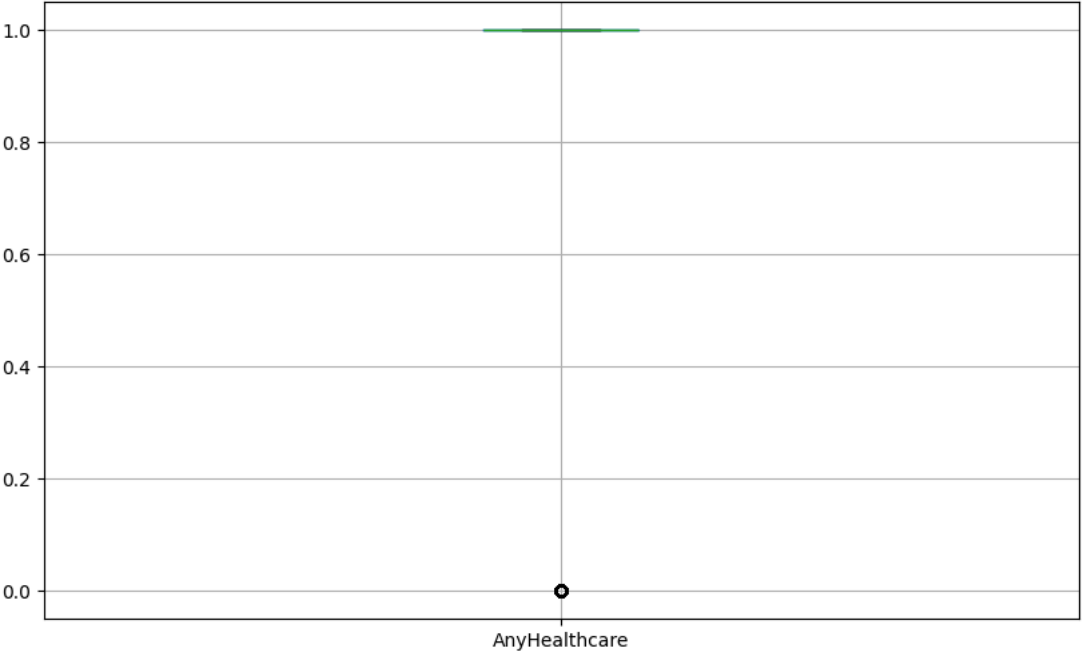




Boxplot of HvyAlcoholConsump

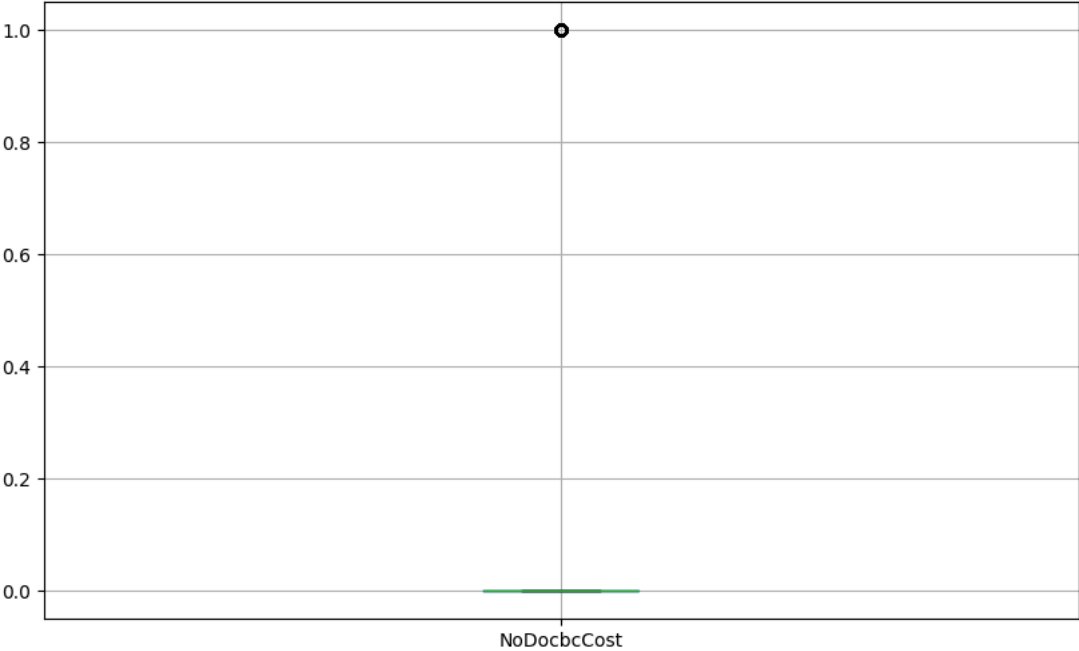


Boxplot of AnyHealthcare

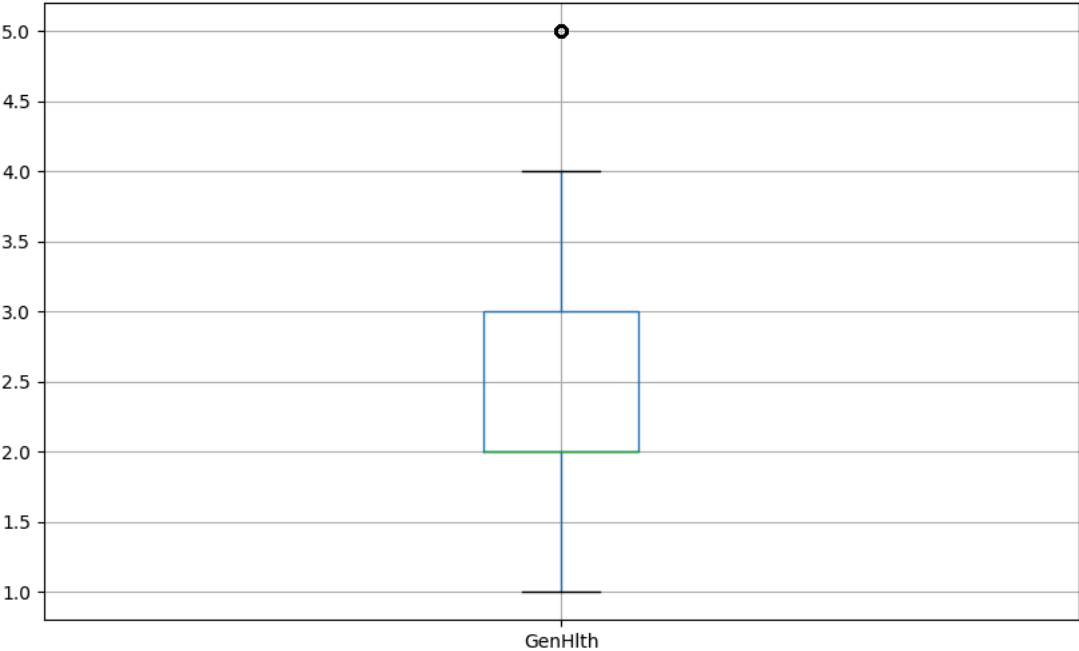




Boxplot of NoDocbcCost

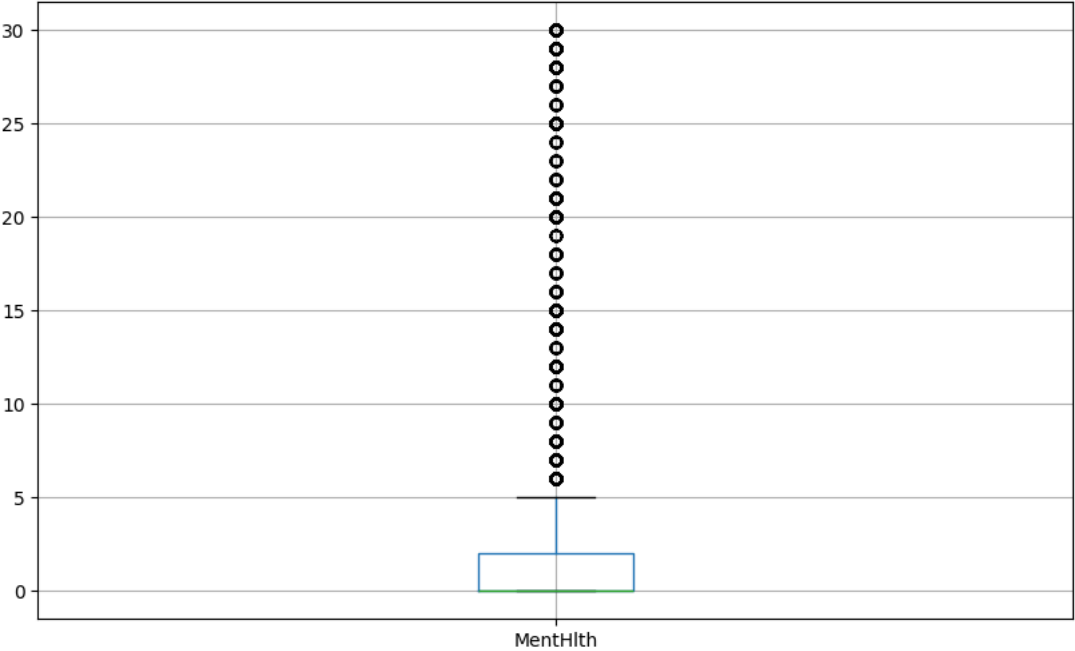


Boxplot of GenHlth

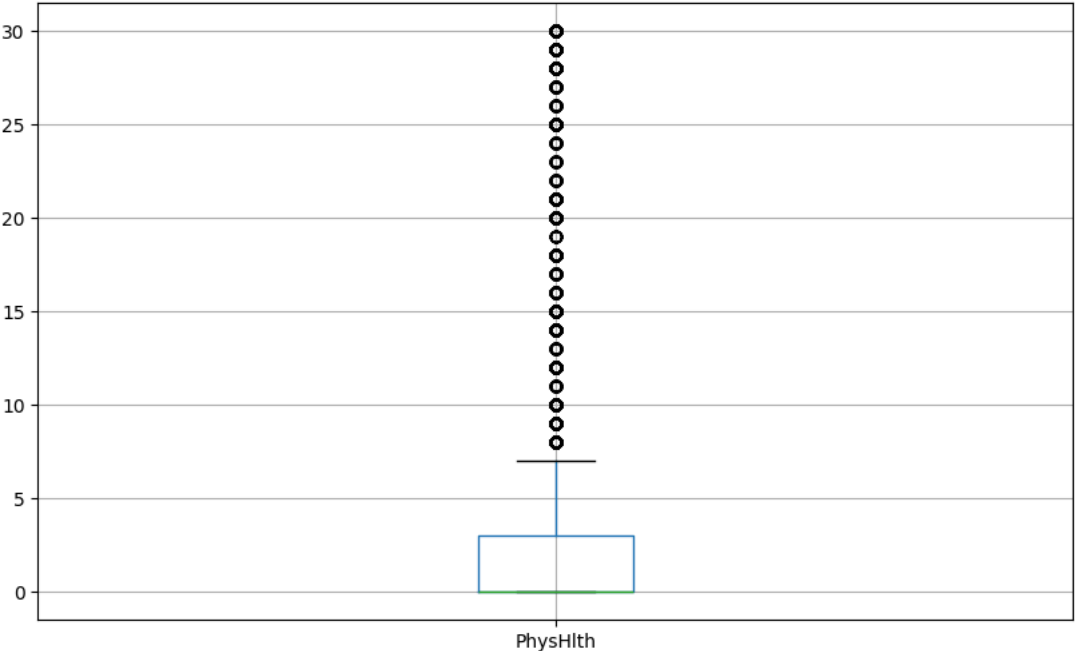




Boxplot of MentHlth

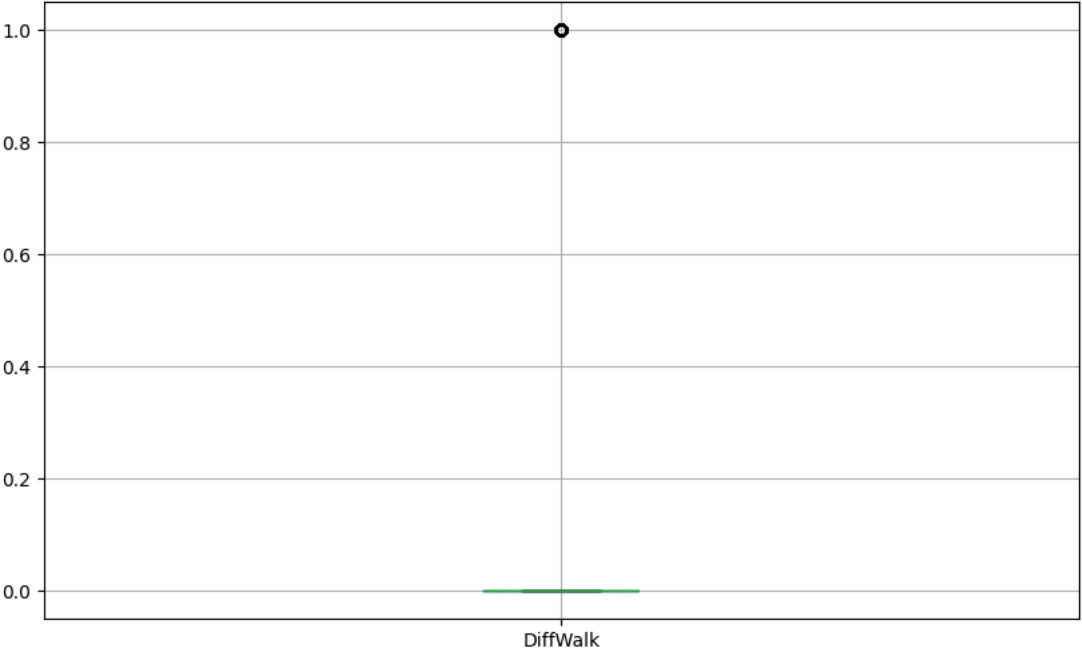


Boxplot of PhysHlth

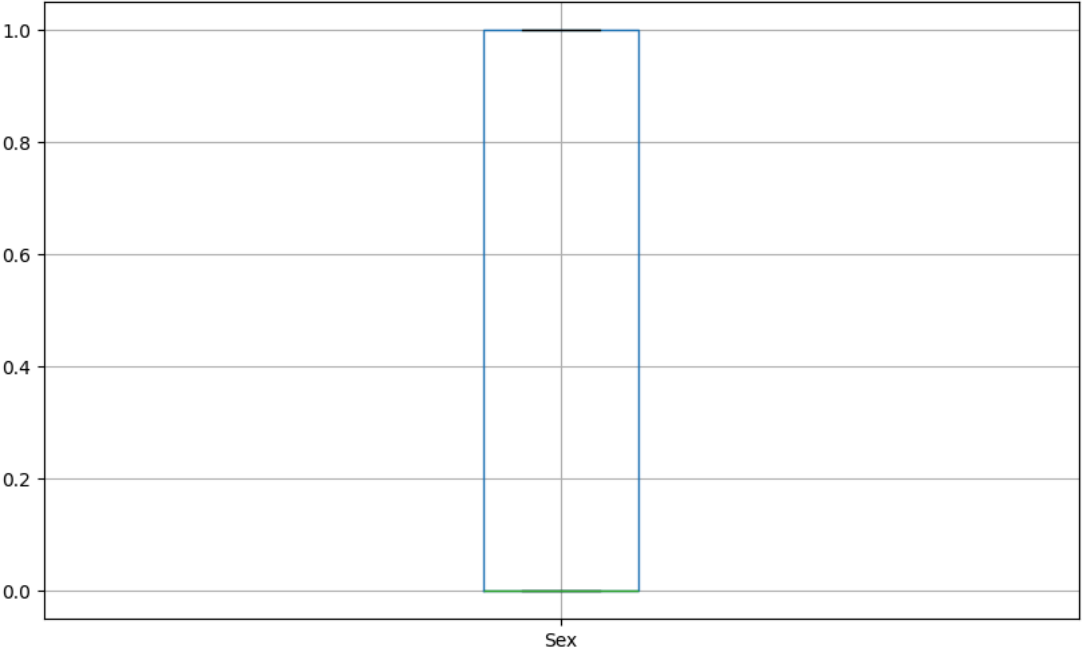




Boxplot of DiffWalk

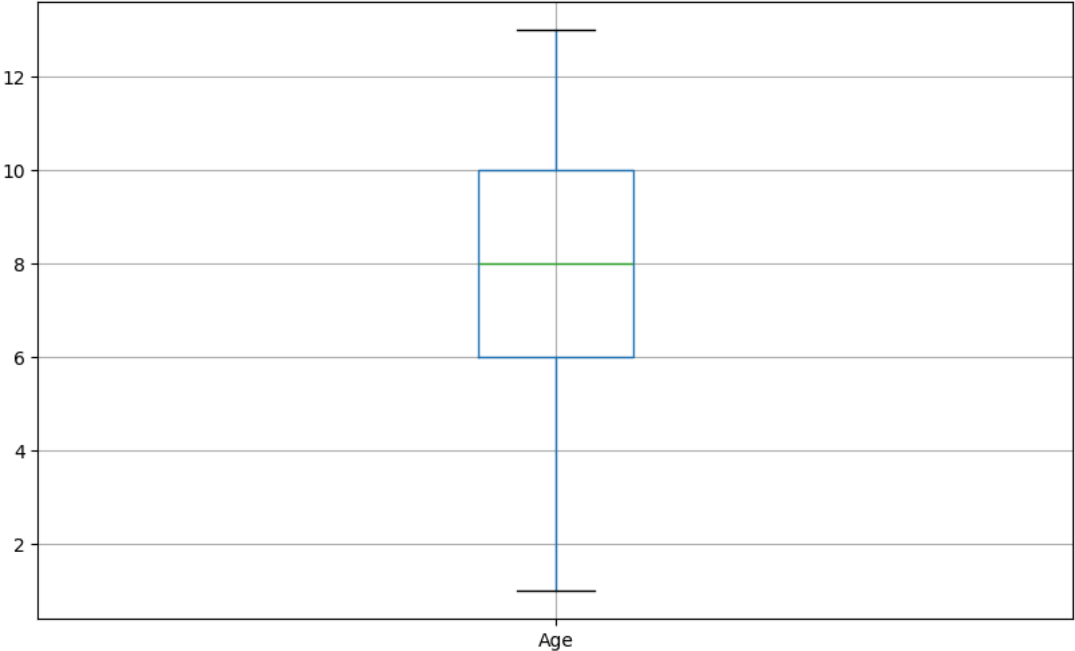


Boxplot of Sex

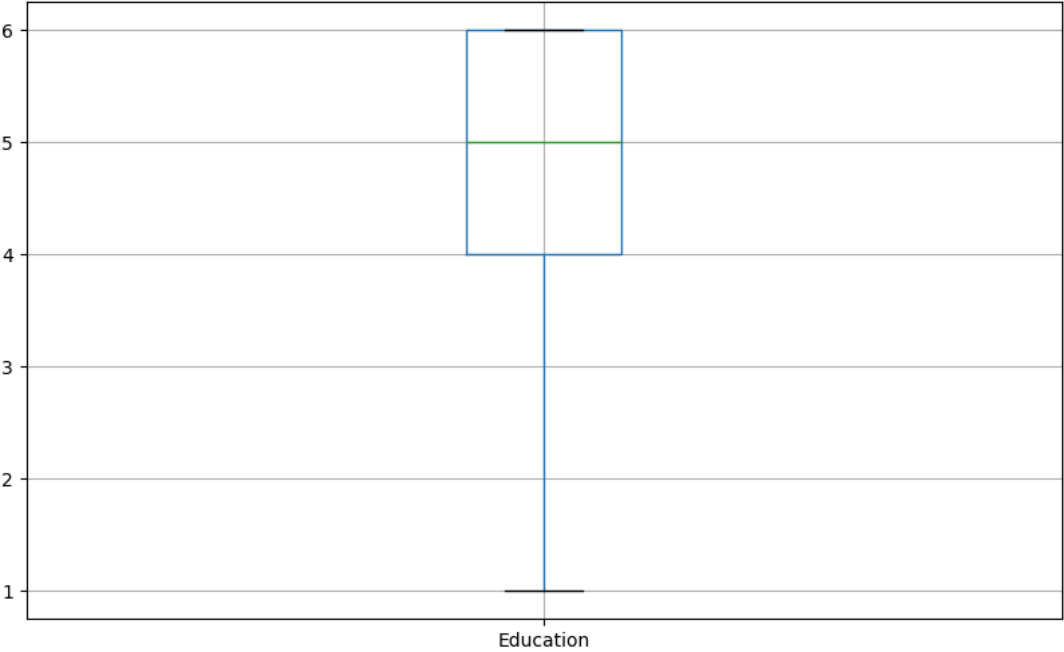


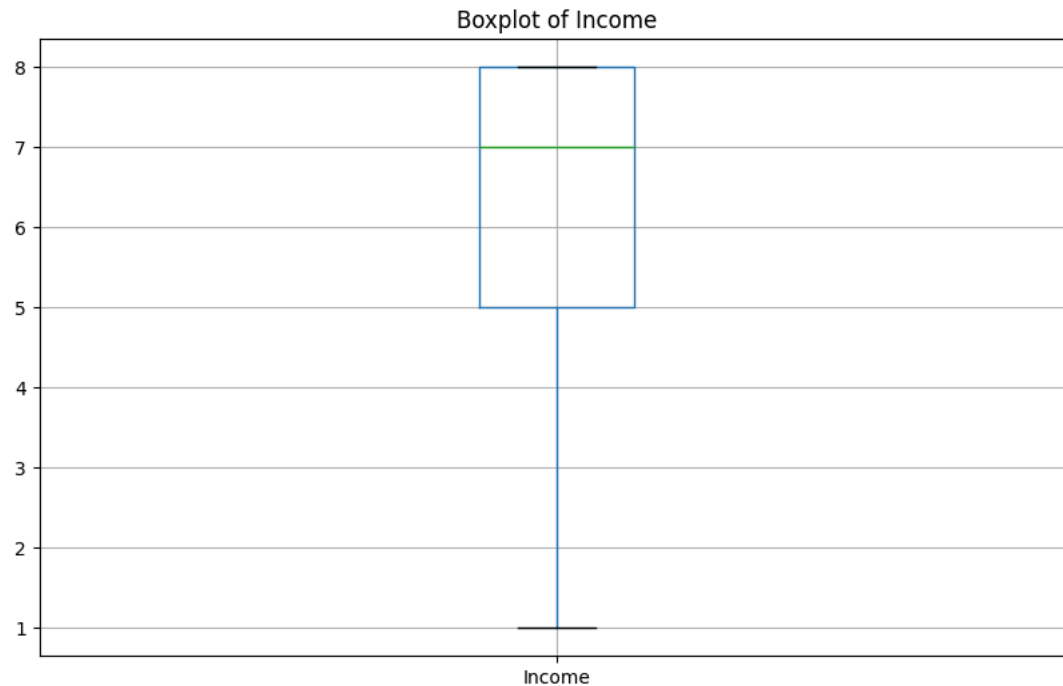


Boxplot of Age



Boxplot of Education





Alternatively, we can use the IQR method to detect outliers.

```
```python
IQR method to detect outliers
for column in numerical_columns:
 Q1 = df[column].quantile(0.25)
 Q3 = df[column].quantile(0.75)
 IQR = Q3 - Q1

 lower_bound = Q1 - 1.5 * IQR
 upper_bound = Q3 + 1.5 * IQR

 display(f'Column: {column}, Lower bound: {lower_bound}, Upper bound: {upper_bound}')
 display(df[(df[column] < lower_bound) | (df[column] > upper_bound)].shape)
```
```

```
'Column: Diabetes_012, Lower bound: 0.0, Upper bound: 0.0'
(39977, 22)
'Column: HighBP, Lower bound: -1.5, Upper bound: 2.5'
(0, 22)
'Column: HighChol, Lower bound: -1.5, Upper bound: 2.5'
(0, 22)
'Column: CholCheck, Lower bound: 1.0, Upper bound: 1.0'
(9470, 22)
'Column: BMI, Lower bound: 13.5, Upper bound: 41.5'
(9847, 22)
'Column: Smoker, Lower bound: -1.5, Upper bound: 2.5'
(0, 22)
'Column: Stroke, Lower bound: 0.0, Upper bound: 0.0'
(10292, 22)
'Column: HeartDiseaseorAttack, Lower bound: 0.0, Upper bound: 0.0'
(23893, 22)
'Column: PhysActivity, Lower bound: 1.0, Upper bound: 1.0'
```



```
(61760, 22)
'Column: Fruits, Lower bound: -1.5, Upper bound: 2.5'
(0, 22)
'Column: Veggies, Lower bound: 1.0, Upper bound: 1.0'
(47839, 22)
'Column: HvyAlcoholConsump, Lower bound: 0.0, Upper bound: 0.0'
(14256, 22)
'Column: AnyHealthcare, Lower bound: 1.0, Upper bound: 1.0'
(12417, 22)
'Column: NoDocbcCost, Lower bound: 0.0, Upper bound: 0.0'
(21354, 22)
'Column: GenHlth, Lower bound: 0.5, Upper bound: 4.5'
(12081, 22)
'Column: MentHlth, Lower bound: -3.0, Upper bound: 5.0'
(36208, 22)
'Column: PhysHlth, Lower bound: -4.5, Upper bound: 7.5'
(40949, 22)
'Column: DiffWalk, Lower bound: 0.0, Upper bound: 0.0'
(42675, 22)
'Column: Sex, Lower bound: -1.5, Upper bound: 2.5'
(0, 22)
'Column: Age, Lower bound: 0.0, Upper bound: 16.0'
(0, 22)
'Column: Education, Lower bound: 1.0, Upper bound: 9.0'
(0, 22)
'Column: Income, Lower bound: 0.5, Upper bound: 12.5'
(0, 22)
We will also check the summary statistics of the numerical columns to understand the distribution.
```

```
```python
Summary statistics for numerical columns
display('Summary statistics for numerical columns:')
display(df[numerical_columns].describe())
```
```

| | Diabetes_012 | HighBP | HighChol | CholCheck | BMI | Smoker | Stroke | HeartDiseaseorAttack | PhysActivity | Fruits | ... | AnyHealthcare | |
|-------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|----------------------|---------------|---------------|-----|---------------|---------------|
| count | 253680.000000 | 253680.000000 | 253680.000000 | 253680.000000 | 253680.000000 | 253680.000000 | 253680.000000 | 253680.000000 | 253680.000000 | 253680.000000 | ... | 253680.000000 | 253680.000000 |
| mean | 0.296921 | 0.429001 | 0.424121 | 0.962670 | 28.382364 | 0.443169 | 0.040571 | 0.094186 | 0.756544 | 0.634256 | ... | 0.951053 | 0.951053 |
| std | 0.698160 | 0.494934 | 0.494210 | 0.189571 | 6.608694 | 0.496761 | 0.197294 | 0.292087 | 0.429169 | 0.481639 | ... | 0.215759 | 0.215759 |
| min | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 12.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000000 | ... | 0.000000 | 0.000000 |
| 25% | 0.000000 | 0.000000 | 0.000000 | 1.000000 | 24.000000 | 0.000000 | 0.000000 | 0.000000 | 1.000000 | 0.000000 | ... | 1.000000 | 1.000000 |
| 50% | 0.000000 | 0.000000 | 0.000000 | 1.000000 | 27.000000 | 0.000000 | 0.000000 | 0.000000 | 1.000000 | 1.000000 | ... | 1.000000 | 1.000000 |
| 75% | 0.000000 | 1.000000 | 1.000000 | 1.000000 | 31.000000 | 1.000000 | 0.000000 | 0.000000 | 1.000000 | 1.000000 | ... | 1.000000 | 1.000000 |
| max | 2.000000 | 1.000000 | 1.000000 | 1.000000 | 98.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | ... | 1.000000 | 1.000000 |

8 rows x 22 columns

These methods will help us understand if there are any outliers in the data.

Before performing the above operations, let's check for null values.

```
```python
Check for null values
display('Display of missing values:')
display(df.isnull().sum())
```
```





'Display of missing values: '

| | 0 |
|----------------------|---|
| Diabetes_012 | 0 |
| HighBP | 0 |
| HighChol | 0 |
| CholCheck | 0 |
| BMI | 0 |
| Smoker | 0 |
| Stroke | 0 |
| HeartDiseaseorAttack | 0 |
| PhysActivity | 0 |
| Fruits | 0 |
| Veggies | 0 |
| HvyAlcoholConsump | 0 |
| AnyHealthcare | 0 |
| NoDocbcCost | 0 |
| GenHlth | 0 |
| MentHlth | 0 |
| PhysHlth | 0 |
| DiffWalk | 0 |
| Sex | 0 |
| Age | 0 |
| Education | 0 |
| Income | 0 |

dtype: int64

